Анализ данных на F#. Part 1

FSharp.Data, XPlot

Никитин Артём 24Б44

30 апреля 2026 г.

FSharp.Data: Введение

```
open FSharp.Data

let apiUrl = "https://api.openweathermap.org/data/2.5/weather?q="

type Weather = JsonProvider<sample>

let sf = Weather.Load(apiUrl + "Saint Petersburg")

sf.Sys.Country
sf.Main.Temp
```

- Ещё есть провайдеры для CSV, HTML, и XML
- Предоставляют простой доступ к структурированным данным

FSharp.Data: CSV

HLOC: High, Low, Open, Close — типичная структура финансовых данных

```csv
Date,Open,High,Low,Close,Volume
9-Oct-17,75.97,76.55,75.86,76.29,11386502
6-Oct-17,75.67,76.03,75.54,76.00,13959814
5-Oct-17,75.22,76.12,74.96,75.97,21195261
4-Oct-17,74.09,74.72,73.71,74.69,13317681
3-Oct-17,74.67,74.88,74.20,74.26,12190403
2-Oct-17,74.71,75.01,74.30,74.61,15304762
29-Sep-17,73.94,74.54,73.88,74.49,17079114
28-Sep-17,73.54,73.97,73.31,73.87,10883787
27-Sep-17,73.55,74.17,73.17,73.85,19375099
```

FSharp.Data: CSV Provider

```
// Тип-провайдер
type Stocks = CsvProvider<
    "../data/MSFT.csv",                          // образец данных
    ResolutionFolder=__SOURCE_DIRECTORY__    // где искать файл
>

// Загружаем и кэшируем данные
let msft =
    Stocks
        // Работает лениво
        .Load(__SOURCE_DIRECTORY__ + "/data/MSFT.csv")
        // Выгузило в RAM весь файл
        .Cache()
```

FSharp.Data: CSV Provider

```fsharp
// Получаем первую строку
let firstRow = msft.Rows |> Seq.head
let lastDate = firstRow.Date
let lastOpen = firstRow.Open

printfn $"Дата: {lastDate}, Открытие: {lastOpen}"

// Выводим первые 10 строк в формате HLOC
printfn "\nПервые 10 записей (High, Low, Open, Close):"
for row in msft.Rows |> Seq.truncate 10 do
    printfn "HLOC: (%A, %A, %A, %A)"
        row.High
        row.Low
        row.Open
        row.Close
```

FSharp.Data: CSV Provider

Что получилось:

```
Last date: 10/9/2017 12:00:00 AM, Open: 75.97
HLOC: (76.55M, 75.86M, 75.97M, 76.29M)
HLOC: (76.03M, 75.54M, 75.67M, 76.00M)
HLOC: (76.12M, 74.96M, 75.22M, 75.97M)
HLOC: (74.72M, 73.71M, 74.09M, 74.69M)
HLOC: (74.88M, 74.20M, 74.67M, 74.26M)
HLOC: (75.01M, 74.30M, 74.71M, 74.61M)
HLOC: (74.54M, 73.88M, 73.94M, 74.49M)
HLOC: (73.97M, 73.31M, 73.54M, 73.87M)
HLOC: (74.17M, 73.17M, 73.55M, 73.85M)
HLOC: (73.81M, 72.99M, 73.67M, 73.26M)
```

FSharp.Data: JSON Provider; Типизация

```
type Simple = JsonProvider<""" { "name":"John", "age":94 } """>
let simple = Simple.Parse(""" { "name":"Tomas", "age":4 } """)
simple.Age
simple.Name
```

```
F# Interactive:
type Simple = JsonProvider<...>
val simple: JsonProvider<...>.Root = {
  "name": "Tomas",
  "age": 4
}
val it: string = "Tomas"
```

FSharp.Data: JSON Provider

```
type Mixed = JsonProvider<""" [1, 2, "hello", "world"] """>
let mixed = Mixed.Parse(""" [4, 5, "hello", "world" ] """)

mixed.Numbers |> Seq.sum
mixed.Strings |> String.concat ", "
```

FSharp.Data: JSON Provider; Работа с GitHub

```
// GitHub.json downloaded from
// https://api.github.com/repos/fsharp/FSharp.Data/issues
type GitHub = JsonProvider<
    "../data/GitHub.json",
    ResolutionFolder=ResolutionFolder
>

let topRecentlyUpdatedIssues =
    GitHub.GetSamples()
    |> Seq.filter (fun issue -> issue.State = "open")
    |> Seq.sortBy (fun issue -> System.DateTimeOffset.Now - issue.Up
    |> Seq.truncate 2

for issue in topRecentlyUpdatedIssues do
    printfn "#%d %s" issue.Number issue.Title
```

```
#879 Bug when call request from Http module
#878 Header being considered as data row in HTMLProvider
```

FSharp.Data: JSON Provider; Создание issue

```
type GitHubIssue = JsonProvider<issueSample, RootName="issue">

let newIssue =
    GitHubIssue.Issue(
        "Test issue",
        "This is a test issue created in FSharp.Data documentation",
        assignee = "",
        labels = [||],
        milestone = 0
    )

newIssue.JsonValue.Request
    "https://api.github.com/repos/fsharp/FSharp.Data/issues"
```

XPlot

- На основе Plotly и Google Charts
- Plotly: Bar, (3D)Line, Area ...
- Google Charts: Calendar, Geo, Pie ...
- Network

XPlot: Bar Char

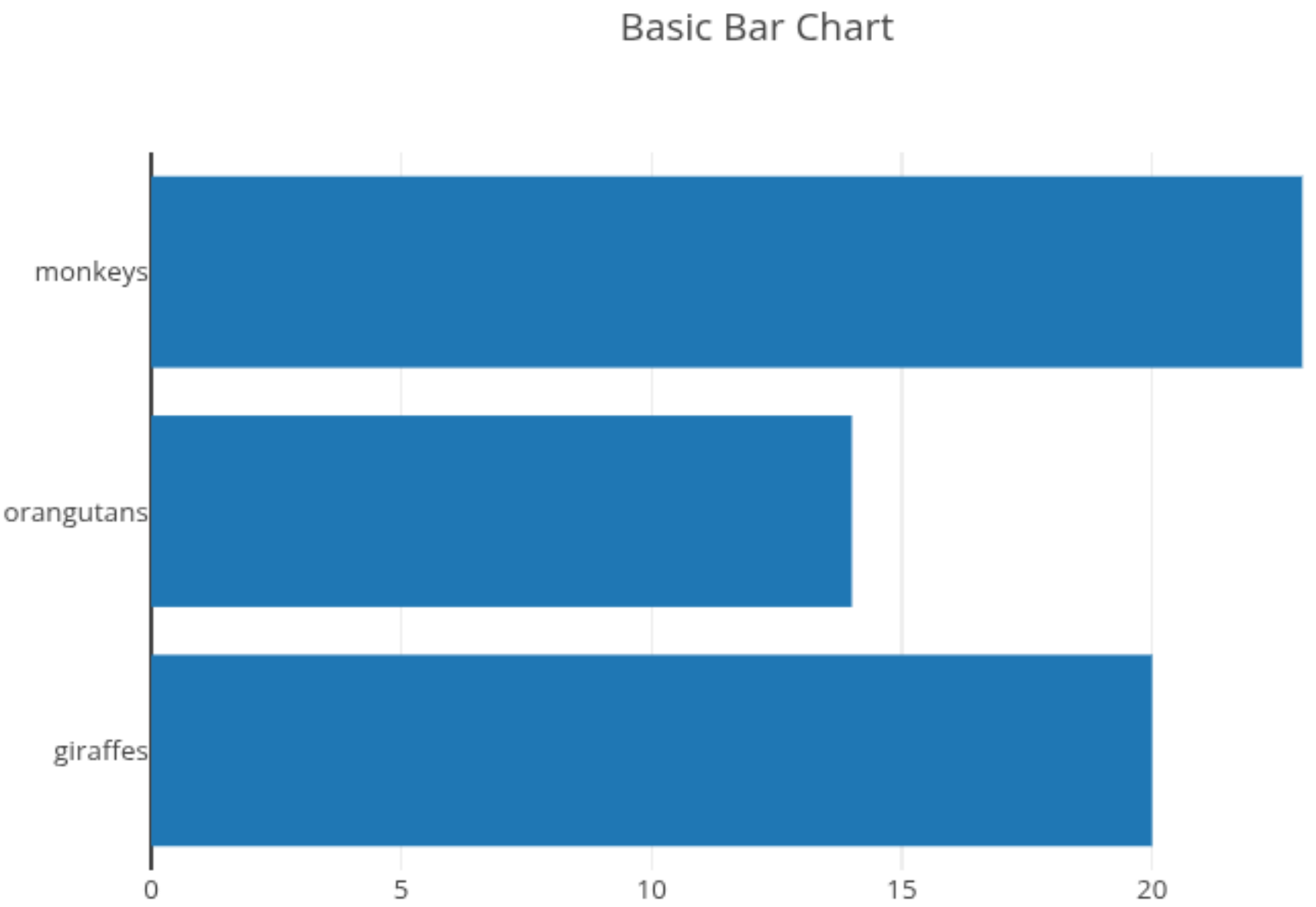
```
open XPlot.Plotly

let layout = Layout(title = "Basic Bar Chart")

let data = ["giraffes", 20; "orangutans", 14; "monkeys", 23]

let chart1 =
    data
    |> Chart.Bar
    |> Chart.WithLayout layout
    |> Chart.WithHeight 500
    |> Chart.WithWidth 700
```

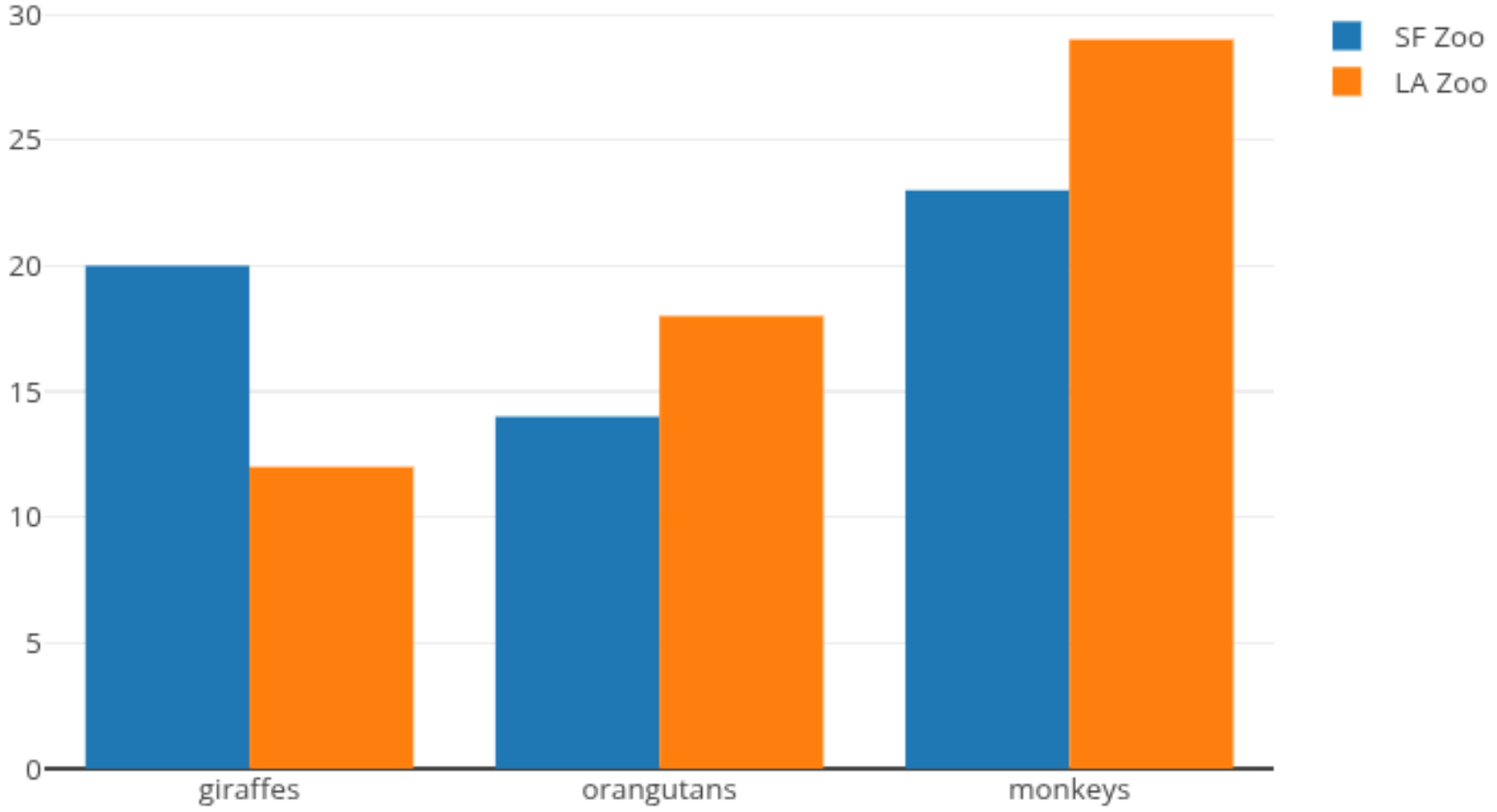
XPlot: Bar Char



XPlot: Grouped Bar Char

```
let groupedTrace1 =
    Bar(
        x = ["giraffes"; "orangutans"; "monkeys"],
        y = [20; 14; 23],
        name= "SF Zoo")
let groupedTrace2 =
    Bar(
        x = ["giraffes"; "orangutans"; "monkeys"],
        y = [12; 18; 29],
        name = "LA Zoo")

let groupedLayout = Layout(barmode = "group")

let chart2 =
    [groupedTrace1; groupedTrace2]
    |> Chart.Plot
    |> Chart.WithLayout groupedLayout
    |> Chart.WithHeight 500
    |> Chart.WithWidth 700
```

XPlot: Grouped Bar Char